AI4Research: Workflow, State of the Art, and the Target Platform

Muk Chunpongtong · September 2026

This report covers 3 subjects:

- How AI4Research turns a research request into checked evidence, and what is still hard
- How that compares with the systems that define the state of the art
- Which platform the work moves to, and what AI4Research adds on top of it

Code claims were read in the source of AI4Research, DeepSeek Harness and openJiuwen at pinned commits, and paper claims were read in the cited paper. This version keeps code paths and line numbers out of the text; the annotated variant, kept beside it, carries the location and excerpt behind every code claim.

Appendix A lists the sources, Appendix B answers the questions a reviewer is likely to ask, Appendix C says where the code evidence is, and Appendix D holds the workflow details that the body leaves out.

1. The goal: 3 claims, and the 6 terms they rest on

The report is written for a reader who builds agents but has not seen this codebase. It makes 3 claims, and Sections 2 to 6 give the evidence for each:

- The workflow has 1 boundary that matters, the freeze, which fixes the plan, the capabilities bound to it and the rules that will grade it before anything runs
- The state of the art generates research well and checks it weakly, while AI4Research appears to do the opposite: it checks strictly and pays for it in code generation, claim recall and external evaluation
- The DeepSeek Harness supplies the runtime that AI4Research built by hand, and the contract layer is what AI4Research adds

AI4Research is a local-first multi-agent system. It compiles a natural-language research goal into a typed, hash-verified contract, then executes that contract as a task graph in which each step runs on an interchangeable operator: a model, a command-line coding agent or a tool.

The project states its target design in 1 sentence, translated here from the Chinese original:

> *LLMs interpret semantics; deterministic code validates structure, maintains references, freezes contracts, and schedules.*

This is a design target, not a measured property, so Section 2 checks, stage by stage, where a model proposes and where code decides.

6 terms carry the whole report (Table 1).

Term	Meaning
capsule	A contract for 1 capability, written in YAML: its typed inputs and outputs, its effects on the system, its preconditions, postconditions and invariants, and how it is verified. The registry holds 80 entries (77 capabilities, 1 guard and 2 resources), and 74 of the 79 capsule manifests are registered
operator	Whatever executes a step. A logical operator is an abstract role with graded requirements, and 45 are defined. A physical operator is a concrete executor, such as a model endpoint, a CLI agent or a tool, and 102 are defined, of which 63 are enabled
sprint	1 request carried from compiled intent to closure

Term	Meaning
node	1 step of a sprint's task graph
gate	A check that decides PASS or FAIL for the output of 1 node
freeze	The point at which the plan, its capability bindings and its grading rules are hashed into 1 run contract

Table 1. The 6 terms the report rests on.

2. The workflow: models propose, code checks, and the freeze fixes the rules

A request passes through the stages in Fig. 1, and the freeze divides them in 2. Before the freeze, a model proposes each artifact and deterministic code checks it. At the freeze, code hashes the plan, its capsule bindings and its grading rules into 1 run contract. After the freeze, no model schedules work, routes it to an operator or admits a result, though models still do the work inside capsules and judge inside rubric gates.

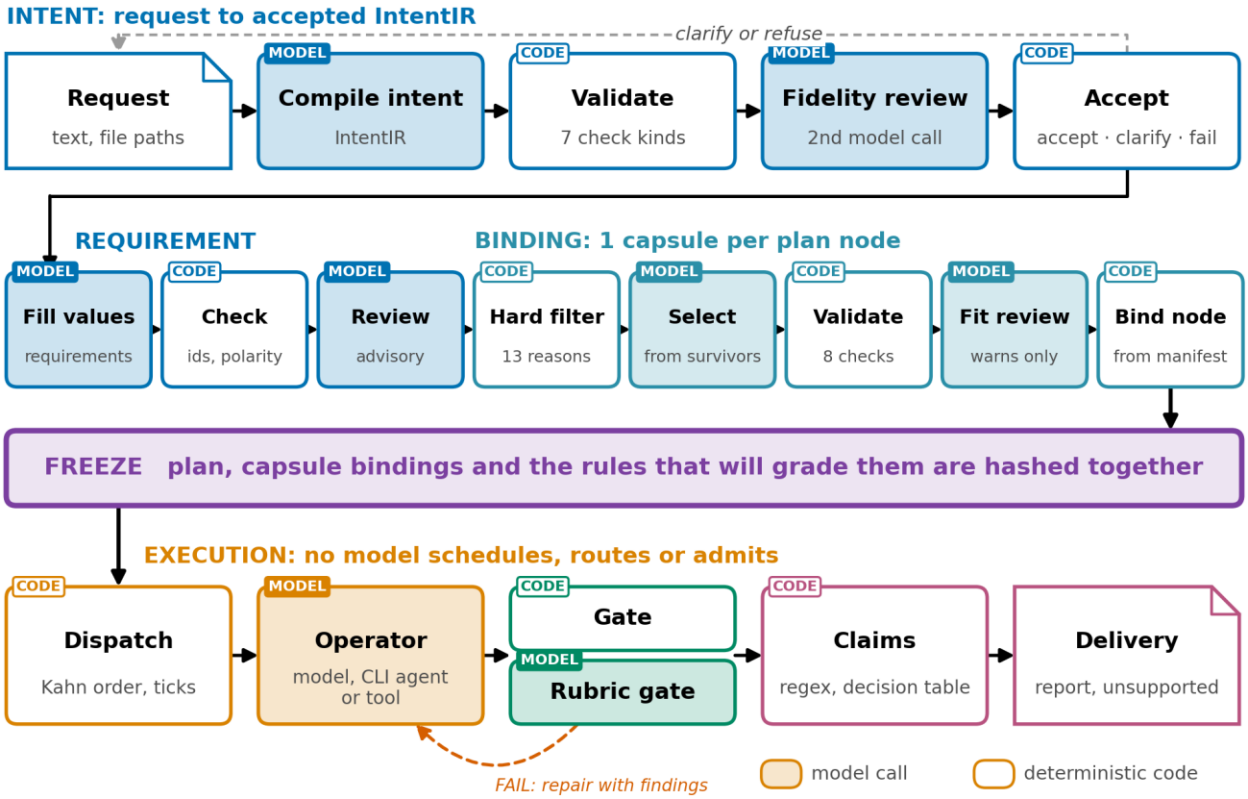


Fig. 1. The workflow. Above the freeze, every model call is followed by code that checks it; below it, a frozen contract is executed. Tinted boxes are model calls, white boxes deterministic code, and colour marks the stage.

Each subsection below says what the stage does, what was found in the source, what could address it, and what other systems do. Appendix D gives the detail of each stage.

2.1 Before the plan: code checks every model output

The first 2 stages turn the request into a structured record of what was asked, the IntentIR, and then into a requirement contract, which states what would satisfy the request before any stage decides how (Table 2).

Step	Author	What it does

Step	Author	What it does
Compile intent	A model	Writes the IntentIR: goals, typed outcomes, constraints, ambiguities, conflicts and unknowns
Validate	Deterministic code	Runs 7 kinds of check, and finds every quote the model gives in the request text
Fidelity review	A second, independent model call	Checks that the IntentIR reflects the request
Accept	A deterministic gate	Accepts, returns a blocking ambiguity or conflict to the user, or fails
Fill requirements	A model	Fills in values only; the valid Intent ids and check ids are fixed lists in the schema
Check requirements	Deterministic code	Requires constraints copied byte for byte, every Intent id covered, and only registered checks

Table 2. The steps before planning, and who authors each.

Code keeps the model in check in 3 ways. Every quote the model gives must be found in the request text, so the record cannot point at text that is not there. Repair is bounded to 1 attempt by default and at most 4, on the reasoning that each extra repair is a chance to fit the validator rather than the request. And the ids a model may name come from fixed lists in the schema, so an invented check fails validation before anything reads it.

The fidelity review has caught a real error at least once. In the 14-node run of 2026-09-03 it caught the compiler strengthening "distinguish 5 evidence statuses" into "only these 5", and 1 repair fixed it.

What was found. The checks registry holds 43 checks, and 20 of them are rubrics judged by a model rather than code; Section 2.6 notes that results do not record which kind decided. A fidelity objection is demoted to a warning once the repair budget is spent, which seems deliberate, so a review model never becomes a second gate, but a meaning-level error the reviewer found can then pass. And 1 run filtered its literature search on the words "one-off answer" and emptied its own shortlist, because corpus-selection criteria are not separated from process language in the free text.

What could address it. Marking on each result whether a rubric or a code gate decided it, and separating corpus criteria from process language in the request schema.

What others do. Research agents such as AI Scientist-v2 (2504.08066) and Kosmos (2511.02824) take a topic straight into an agent loop and leave no typed artifact a later stage can check. The closest published design seems to be PlanCompiler (2604.13092), where the model selects typed nodes from a closed registry and static checks validate the graph; it scores 44 of 50 on schema-trap tasks against 20 of 50 for GPT-4.1 (p1), but it has not been applied to research workflows.

2.2 Binding: code narrows the capsules, a model picks, and code binds

The plan says how the request will be met and which capability each step uses (Fig. 2). Before any model sees the candidates, a hard filter removes capsules for 13 reasons: wrong input or output types, an unsatisfiable proof class, effects that do not match the node's request in both directions, network rules, a missing verification contract, an undeclared implementation and unsatisfied trust. The planner model picks among the survivors, code validates the choice 8 ways, a second model reviews the fit but can only warn, and what is bound is copied from the manifest rather than from the model. Appendix D.3 lists the 8 steps of planning.

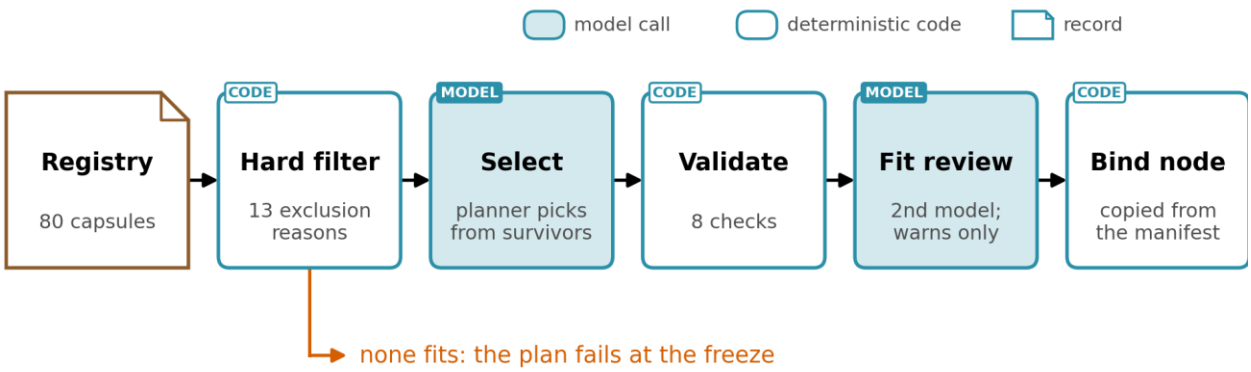


Fig. 2. Capsule binding: code filters, validates and binds; a planner model selects among the survivors and a second model reviews the fit, advisory only. When no capsule fits, the plan fails at the freeze.

What was found. There is no fallback when nothing fits: a node no capsule or composition can serve is marked unsatisfiable, and the whole plan fails at the freeze. Capsules that call models are opaque at planning, since the planner sees only their trust class, reasoning class and operators. The fit reviewer may be the same model as the planner, and the record says so when it is.

What could address it. The companion report "Capability Capsule in AI4Research", called the capsule report throughout, proposes a generalist capsule for the nodes nothing fits, with a record of each gap.

2.3 The freeze: the plan and its grading rules are hashed before anything runs

The freeze turns the negotiated plan into a contract in 8 steps of deterministic code, and any step can fail the sprint (Appendix D.4). The evaluation budget becomes a test policy first, so the grading rules exist before anything is sealed. If a capsule or operator definition changed during planning, the freeze aborts.

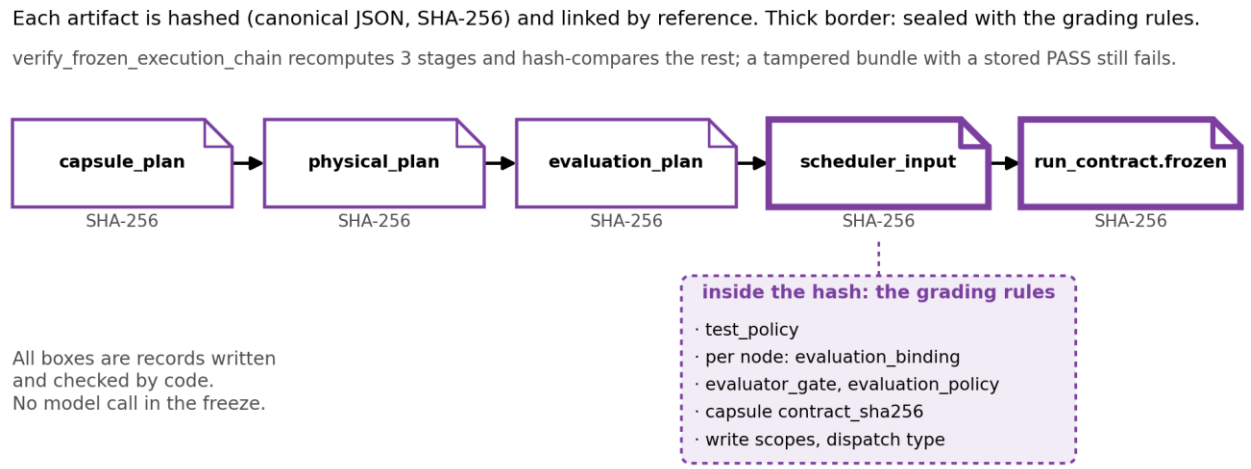


Fig. 3. The freeze chain. Each artifact is hashed and linked by reference; thick borders are hashed with the grading rules inside them.

The grading rules are inside the hash. Each node's evaluator bindings and the run's test policy are written into the scheduler input, and the hash covers them. If they sat outside it, a run could be judged by rules chosen after its outputs were seen. An existing test sets a node's priority to 999 in a frozen bundle, and verification fails even though the stored verdict still says PASS. A hash detects an edit after the fact; it prevents none and says nothing about who made it.

What was found. There is no re-planning: when the freeze fails, or when execution later shows the plan was wrong, the sprint ends and the user resubmits. Search happens only inside operators. Plan defects per run are not measured. An open capability such as a shell or SQL can route around typed contracts, which PlanCompiler calls "constraint evasion under partial enforcement" (p3) and which caused 13 of its 22 failures (p17). And a frozen rule carries no branch for an outcome it did not anticipate: at every risk tier the budget gives a model evaluator to 1 node and freezes every other node with no evaluator gate, and such a node records a policy pass.

What could address it. Counting plan defects per run before deciding whether the freeze keeps its cost, and letting the test policy say what a node does when its rules do not cover the outcome: halt, return inconclusive, or ask a person.

What others do. Some work fixes parts of a run before execution, but none of the reviewed systems appears to fix the grading rules with the plan (Appendix D.4 compares them). A targeted search on 2026-09-13 found no reviewed system that seals its own evaluation criteria inside the run hash, though the search may be incomplete. The survey "Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap" (2608.05179) recommends something close from outside the run, "a pre-committed task specification that fixes the metric, the split, and the success threshold before the run" (p28). Arbor (2606.11926) and Kosmos take the opposite trade and re-plan every cycle (Section 4).

2.4 Dispatch: a failure stays contained, and gates never move

Execution runs in ticks. The scheduler repeatedly picks the nodes that are ready and dispatches them, a separate ledger holds everything that changes during the run, and every step of a tick is deterministic code (Appendix D.5). For every node, the freeze fixes the bound capsule and its contract hash, the operator candidates, the evaluator gate, the write scope, the repair budget and the failure policy, and the runtime refuses any change to a frozen field.

Fig. 4 follows an illustrative graph of 7 nodes through 4 ticks. On each tick the scheduler orders the nodes topologically, keeps those whose dependencies have all passed, takes up to 8 by priority, and holds back any node whose write scope or effects conflict with one already selected. A node that fails its gate is repaired with the gate's findings; once its repairs are spent it is marked failed and its dependents are skipped, while a node waiting for human review holds its dependents pending. A passed node never regresses, and work envelopes are fingerprinted so a retry never repeats a paid call.

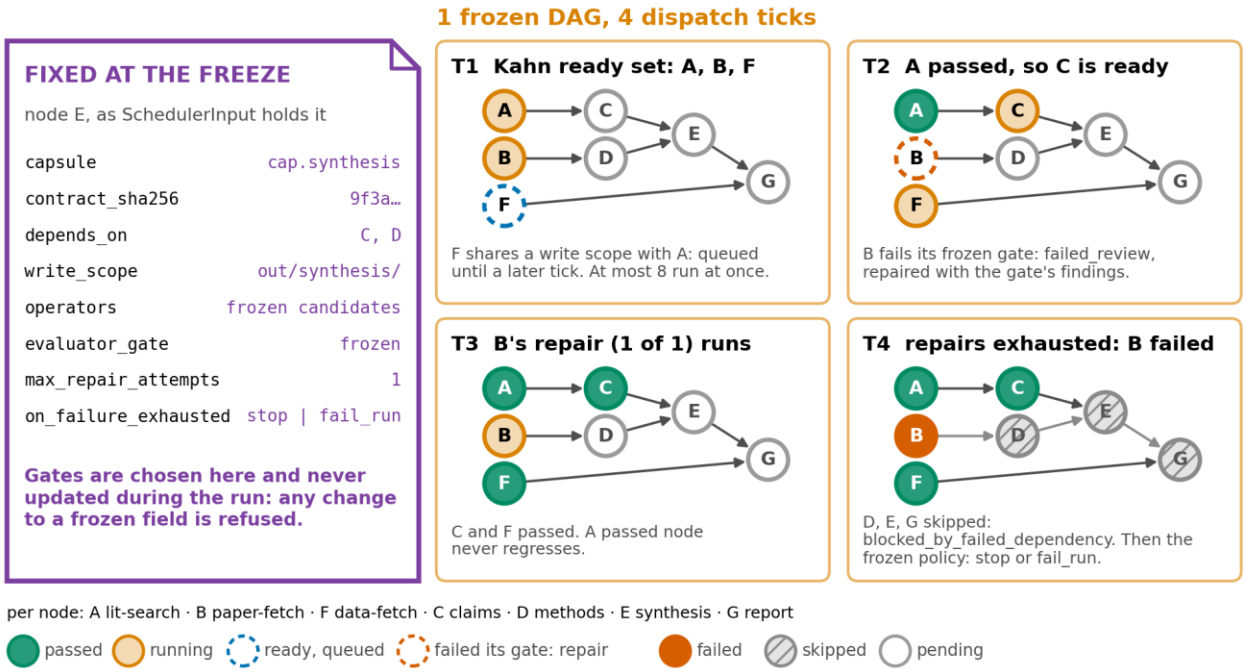


Fig. 4. 1 frozen graph over 4 dispatch ticks: Kahn ready sets, a write-scope conflict, a failed gate with repair, and the skipped dependents of a node whose repairs ran out.

What was found. The substrate is the weak point. Work moves through tmux keystroke injection, file inboxes and SQLite, and that substrate produced failures of its own: 1 coordinator ran code 28 hours older than the file on disk, and a run manifest once recorded the statuses a model declared rather than those in the ledger, a defect a deterministic materializer has since fixed.

What could address it. The move to the DeepSeek Harness (Section 5), whose services would replace the file inbox and record what runs.

2.5 Operator execution: 1 path is recorded, and the CLI-agent path is not observed

An operator runs a node along 1 of 2 paths (Appendix D.6). On the registry path, code re-verifies the frozen contract before running and keeps the request, schema, response, events and hashes of each model call. On the CLI-agent path, the agent receives a terminal pane in a git worktree, and its commands, files and network access are not observed. Its evidence is classed fixture-or-adapter-only, the class for evidence that did not come from a measured execution.

The trust model follows from these 2 paths (Fig. 5). The user's request is trusted. Papers, web pages, tool output, code a model writes and remote agents are not. The freeze fixes the plan before untrusted content is read, and nodes read it afterwards.

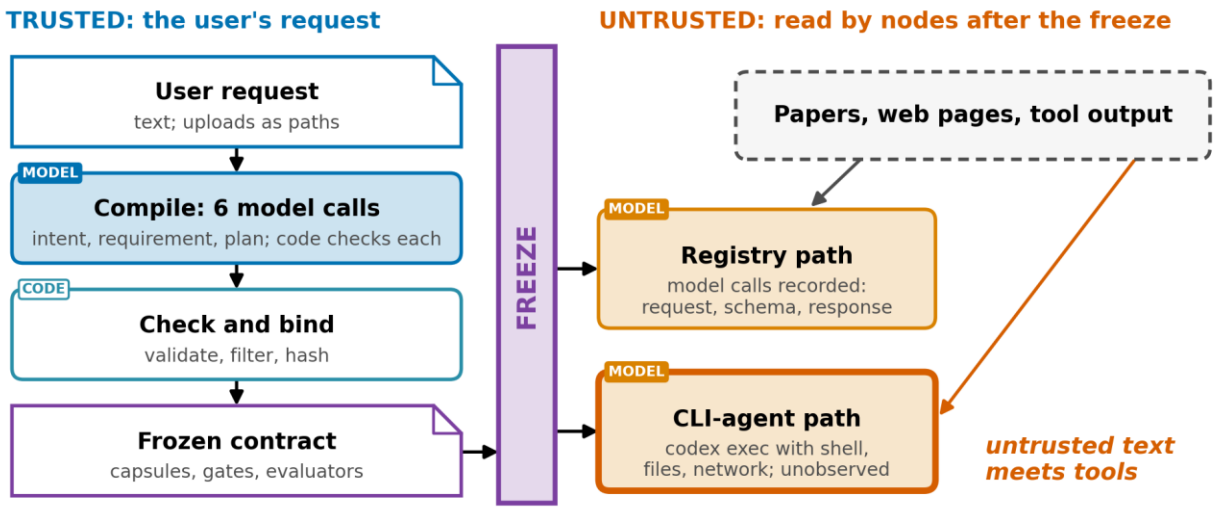


Fig. 5. Trust boundary: the plan is fixed before untrusted content is read; the CLI-agent path hands that content to an agent with tools.

What was found. The CLI-agent path breaks that separation, since the coding agent reads papers and data while holding shell, file and network access, unobserved. A fixed plan also does not stop untrusted text from steering values: a sentence planted in a paper passes the literature check as partially supported (Section 2.7). And the trust model holds for 1 user on a local machine; a shared, unattended server with several users and API credentials would need a different one (capsule report, Appendix G.8).

What others do. CaMeL (2503.18813) fixes control flow before untrusted data is read, the property the freeze also has (p1), and gives untrusted text only to a model with no tools, which the CLI-agent path does not.

2.6 Gates and delivery: code or a model decides, and results do not say which

Every node closes on the evaluators frozen for it, and there are 4 kinds (Table 3). A receipt is not evidence: a node closes only on the chain of result, callback and an evaluator PASS. At the end of the sprint, a terminal barrier publishes the outputs byte-identical to the working directory and closes the sprint.

Kind	Decides by	Note
Deterministic command	Exit code, decided by code	Passes on exit code 0; a timeout counts as an infrastructure failure
Model evaluation	A separate evaluator actor	A model judgment
Code task	A patch, test evidence and a verifier decision	The writer may not be the verifier
Scientific gate	The artifact type	Applied according to the artifact type

Table 3. The 4 kinds of gate.

What was found. A rubric PASS is a model judgment recorded by code, and the result vocabulary does not distinguish the 20 rubrics from the 23 Python gates. The budget is also narrower than that count suggests: at every risk tier 1 node of the graph carries a model evaluator, and every other node is frozen with no evaluator gate, which records a policy pass and leaves the node's proof obligations and artifact schema to decide.

What could address it. Recording on every verdict which kind of check decided it.

2.7 Claims: the check proves where a sentence came from, not that it is true

Claims move through a chain of 6 evidence schemas, from literature discovery to the scientific report, and no model is called in extraction or in the verdicts. A sentence of 30 to 1,000 characters becomes a claim if it contains a cue word such as show, improve, reduce, outperform or found, matched by prefix, so "founded" counts as "found". A claim is testable if it contains any digit, a percent sign, "compared" or "than", so a year qualifies. Each claim carries its citation spans: the offsets into the section text, the quote and the hash of the whole section. If no sentence matches, extraction fails rather than inventing a claim.

Verdicts come from a decision table checked from top to bottom (Fig. 6). When a claim has no experiment, the only evidence is the literature, and the check is a case-insensitive substring match against the cited section, which yields the class source-reported.

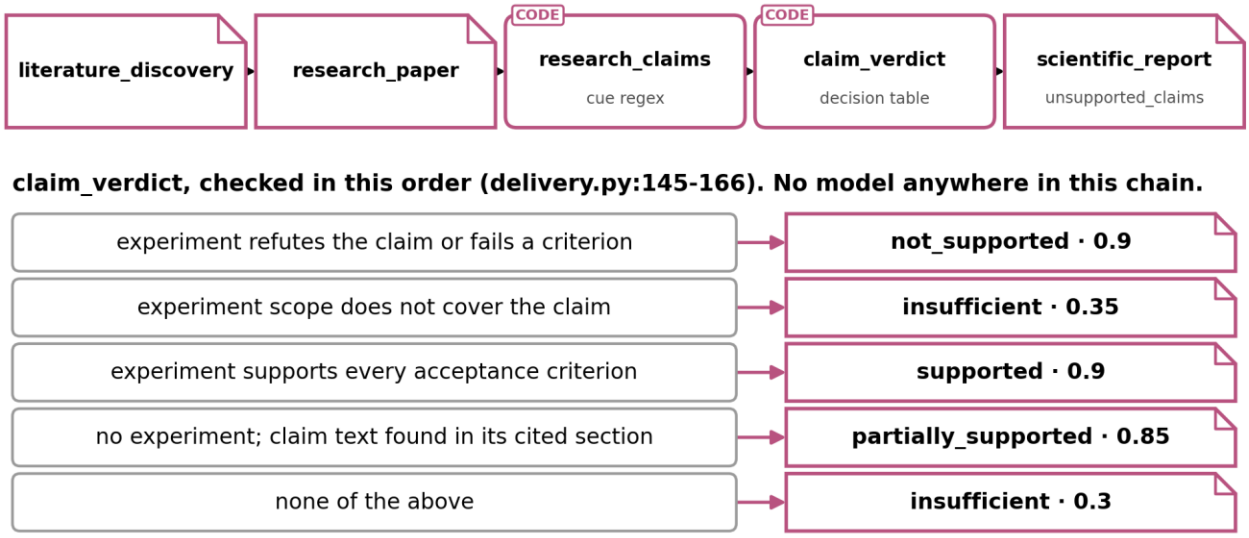


Fig. 6. The claim chain and the verdict decision table, checked top to bottom.

1 claim from a test fixture shows the whole chain (Table 4).

Step	"Reranker A improves ranking quality over reranker B."
Cue word	"improves" matches
Testability	Set to testable in the fixture; the extractor would assign partially testable, since the sentence has no digit, percent sign, "compared" or "than"
Span	The whole sentence, in the results section of the reranker paper, with the hash of that section
Experiment	None
Substring check	Found, case-insensitive
Verdict	Partially supported, class source-reported, not locally reproduced
Gate	No upgrade is possible; the report carries the claim as a conclusion whose evidence status is source-reported
If the span had failed its re-hash	The verdict would be unchanged

Table 4. 1 claim from a test fixture, end to end.

What was found. Without an experiment, the strongest verdict certifies only that the sentence occurs in the cited section, so a wrong, retracted or cherry-picked paper passes. The system records this in 3 structured fields, but whether the rendered report prose shows those labels has not been checked. Spans are re-verified, and a span that fails is dropped silently; a flag turns false only when every span fails, and no gate reads it. The report's list of unsupported claims filters on evidence ids rather than verdicts, so a claim

judged insufficient still becomes a conclusion and the list stays empty. Extraction recall is unmeasured, every claim is typed as a result, multi-clause sentences are not decomposed, and the rules are English only.

What could address it. A gate that fails a claim whose span fails its re-hash, filtering the report on the verdict, and a schema-bound model extractor whose every field carries a verbatim quote that code binds to offsets and hashes, tested per language against a hand-verified claim list.

What others do. CREDENCE (2606.19819) prices rule-based extraction: its rule-only baseline reaches 0.811 Semantic-F1 against 0.88 to 0.91 for model decomposition, because "rule-based splitting cannot resolve complex multi-clause structures" (p22-23), and the cue-word rule here is cruder than that baseline. FactReview (2604.04074) is the closest verdict design, with 4 outcomes and an inconclusive verdict whenever the link between claim and evidence cannot be established (p2); it extracts claims with a model at 79.7% recall (p5). Appendix D.7 has the measured rates of citation and claim failure that make grounding matter.

2.8 Experiments: trusted measurement covers only 1 hand-written family

Experiments are the only place where AI4Research measures rather than cites. 1 capsule carries the trust class measured execution, and it runs only an exact runner and input files whose hashes are bound into the approved plan, with no network, inside the approved write scope, after a hash-bound approval for any run that is not a fixture.

What was found. On the native path, only 1 experiment family exists, KV-cache quantization for causal language models, and its runner was written by hand. On the adapter path, the experiment is delegated to a coding agent and its evidence is classed fixture-or-adapter-only. Only 21 of the 79 manifests declare a trust class at all. So the native path cannot fabricate a measurement but cannot test a claim outside 1 family, while the adapter path has the generality and none of the trust.

What could address it. Letting a model write runners on the native path, where nothing runs until it is hashed and approved.

What others do. Code generation is at the centre of every end-to-end system in Section 4. AI Scientist-v2 writes and runs code at every node of a tree search, and its checking missed a 57% overlap between training and test data (Section 4.1).

3. Evaluation: 1 internal run executed end to end, and no external score yet

3.1 Evidence so far: demonstrated end to end, not yet routinely unattended

The workflow has been demonstrated end to end, but it does not yet appear to be routinely reproducible without a person attending. The evidence comes from the stabilization log and the journey test report (Table 5). Journey tests are the only acceptance evidence; regression suites locate faults but never decide the verdict (Appendix D.8).

Evidence	Result
14-node research run, 2026-09-03	1 bounded intent repair, then the requirement contract and the plan were accepted the first time; every node passed on its first attempt. The run executed its 14 nodes, delivered the 8 requested files and closed with every required gate passed

Evidence	Result
Pre-release regression, 22 affected test files	606 passed, 34 failed and 8 skipped; the 34 failures are kept as known debt rather than relaxed away
Journey tests	24 real tasks run through the production entry points, the only acceptance evidence
External benchmark scores	None; an adapter for Terminal-Bench 2.0 exists

Table 5. Evidence so far.

Results use 6 values, and NOT_TESTED and ENVIRONMENT_BLOCKED are not FAIL. No source records the token cost, the money or the wall-clock time of a sprint, so this report gives no cost figure.

3.2 TruthInsightBench: the first external test, reported per dimension

The proposed first external test is TruthInsightBench (2609.05079), 40 blind discovery tasks drawn from 40 peer-reviewed studies across 10 domains (p1). It looks like the right first test for 3 reasons (Fig. 7). Its scored object is verification: 100 points over 6 dimensions, with evidence auditability 45, robustness 15, control testing 15, cross-dataset generalization 10, novelty 10 and falsifiability 5 (p8). The 4 measured agents score 78% to 81% on evidence auditability and 83% to 87% on novelty, against 11% to 16% on control testing, the dimension that asks whether a simpler explanation would have produced the same result (p14). And its result is a flat plateau, 58.4 to 60.3 with the smallest p = 0.118 (p1, p12), so any real gain would be visible. A frozen model judge applies the rubric, and no instance is graded by a human (p2, p9).

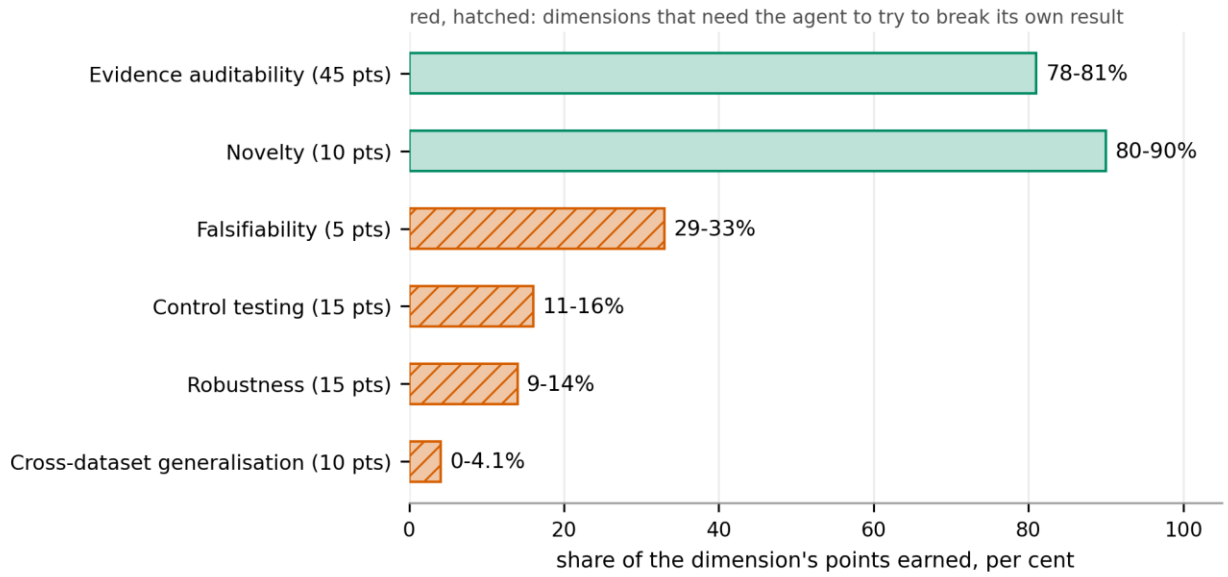


Fig. 7. TruthInsightBench, range across 4 coding agents per dimension (2609.05079 p3, p14). Red, hatched: dimensions that need the agent to try to break its own result.

Results on this benchmark would be more informative per dimension than as a total.

4. The state of the art: 4 reference systems that generate well and check weakly

The 4 systems were chosen to span the design space: generation first (AI Scientist-v2), scale (Kosmos), a mechanical held-out check (Arbor), and the same thesis as AI4Research inside openJiuwen (auto_research).

4 more were read and left out, since the strongest check each relies on is one AI4Research has no path to: Co-Scientist (2502.18864) validates hypotheses in vitro, Robin (2505.13400) confirms a drug candidate in the laboratory, Agent Laboratory (2501.04227) takes human feedback at every stage, and "Towards End-to-End Automation of AI Research" (2606.15497) repeats AI Scientist-v2's workshop experiment.

Each system is placed on the verification ladder of the survey "Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap" (2608.05179). The survey classifies "checks, not systems" (p48), so the placements are this report's own reading. 5 rungs are used (p44): II, an executable test against held-out ground truth; IV, grounding in citations; V, a threat-to-validity argument or a proxy reward, meaning a check the system makes for itself when no independent oracle exists; VI, judgment by a human expert; and VIII, the model's own judgment. Rung V carries most of the weight, because running code the system wrote against tests the system chose is a proxy reward rather than a held-out test.

4.1 AI Scientist-v2: strong generation, and a review that missed a 57% data overlap

AI Scientist-v2 (2504.08066) generates research first and checks it last: ideation with a novelty check, an experiment manager that runs 4 stages with stopping criteria, a tree search inside each stage in which the model writes and runs code, an error marks a node buggy and a vision-language model critiques figures, and writing in 1 pass with a reflection stage (p4-8). 3 manuscripts were submitted blind to an ICLR 2025 workshop, and 1 scored above the acceptance threshold before being withdrawn by prior agreement (p9).

The checking missed a serious flaw. The authors' own post-review examination found about 57% overlap between training and test data in the code (p11), which the workshop reviewers did not raise (p2).

Compared with AI4Research, it searches adaptively at every stage and has no contract, and its last check before submission was human peer review. It seems strong at generating, and its checking is its own execution, peer review and model judgment (rungs V, VI and VIII).

4.2 Kosmos: research at scale, least accurate on synthesis at 57.9%

Kosmos (2511.02824) is built for scale. A scientist supplies an objective and a dataset, each cycle launches up to 10 parallel data-analysis and literature-search tasks, their outputs are summarized into a shared world model that proposes the next cycle, and a run takes over 200 rollouts, writes 42,000 lines of code and reads 1,500 papers (p1-2). Every statement in the report cites a publication or a notebook (p2).

Experts who saw neither the code nor the cited papers rated 79.4% of 102 statements from 3 reports accurate: 85.5% of data-analysis statements, 82.1% of literature statements, and 57.9% of synthesis statements (p2, p4).

What this suggests for AI4Research is the 57.9%: accuracy was checked after the run, and 4 in 10 synthesis statements were wrong. That supports gating claims during the run and keeping claims separate from synthesis, as AI4Research does, though it says nothing about AI4Research's own synthesis, which nothing has measured. Kosmos re-plans its world model every cycle, the opposite of a frozen contract, and its checks are citation grounding and its own execution (rungs IV and V).

4.3 Arbor: a held-out merge gate the merge tool alone can open, the closest system on checking

Arbor (2606.11926) separates a long-lived coordinator, which owns a hypothesis tree, from short-lived executors, each testing 1 hypothesis in its own git worktree (p6-7). An executor may repair the implementation but not change the hypothesis (p8). A held-out merge gate evaluates the best candidate on

the test evaluator in a fresh worktree and merges it only if it improves on the best artifact, and the merge tool "is the only path through which a candidate branch can be promoted" (p8, p28).

On Terminal-Bench 2.0, with 36 development and 53 held-out tasks and Claude Opus 4.6 in both roles, Claude Code has the best development score (75.00) and drops to 71.70 held out, while Arbor scores 72.22 on development and 77.36 held out (p8-10). 2 things are unclear in the paper: how often the test evaluator runs, since the algorithm, the reported protocol and the implementation describe it differently (p7, p26, p29, p33), and whether the number the gate compares was produced by a model, since a secondary model call extracts the metric when the output holds no valid JSON (p30).

Compared with AI4Research, Arbor appears to be the closest system on checking. Its gate is fixed outside the agent it judges, but it is not hashed with a plan, and the plan changes every cycle (rung II).

4.4 openJiuwen auto_research: the same thesis, with the contract renegotiated every round

openJiuwen's agent-core contains an automated research package, auto_research: a module chain from topic survey through experiment design, code, execution and reflection to a LaTeX report; a manager that emits 1 decision each round, execute a subtask under a contract, done, or blocked; host-side validation that rejects illegal moves in code; budgets in the shipped config of 50 rounds and 4 code retries, with lower code defaults behind them; a claim-status classifier that is pure host code and is called nowhere; and paper scoring by a dual rubric. The repository publishes no evaluation of the package.

Compared with AI4Research, it shares the thesis that the model decides and the host validates, but it renegotiates the contract every round rather than freezing it per run, which makes it the sharpest contrast in this report.

4.5 Side by side: AI4Research checks the most and generates the least

System	Plans how	Checks with	Rung	Headline result
AI Scientist-v2	Tree search per stage	Execution; model and vision-language reviewers	V, VI, VIII	1 of 3 workshop papers above threshold; 57% overlap missed
Kosmos	World model, re-planned each cycle	Citation to a notebook or a source	IV, V	79.4% of statements accurate; 57.9% on synthesis
Arbor	Coordinator-owned hypothesis tree	Held-out merge gate, enforced by the merge tool	II	Terminal-Bench 2.0 held out: 77.36 against 71.70 for Claude Code
openJiuwen auto_research	Manager with a contract per round	Host validation each round; rubric scoring	V, VIII	No published evaluation
AI4Research	**Compiled, frozen contract**	**Grading rules hashed with the plan; 23 code gates, 20 rubrics**	**IV, V**	**14 nodes executed and 8 files delivered with every required gate passed; no external score**

Table 6. The 4 reference systems and AI4Research side by side. Every result except openJiuwen's is the authors' own measurement, and openJiuwen's row is read from code. The rung column is this report's own reading; the survey classifies checks rather than systems (2608.05179 p48).

The field is ahead on autonomy, scale and generality. Kosmos reads 1,500 papers in a run, AI Scientist-v2 writes and runs its own experiments, and Arbor's held-out gate, which only a tool can open, seems to pay off. AI4Research's claims are auditable, through frozen gates, retained spans and a verdict on every

conclusion, but they have not been checked externally, so the comparison rests on design properties until they are.

5. The target platform: the DeepSeek Harness supplies the runtime, not the contract

The platform decision of 2026-09-11 was the DeepSeek Harness (dsh) for now, as the more certain of the 2 candidates and already the target of the capsule report. openJiuwen, the other candidate, comes later (Section 5.4).

5.1 Why the harness matters: it moves pass rates almost as much as the model

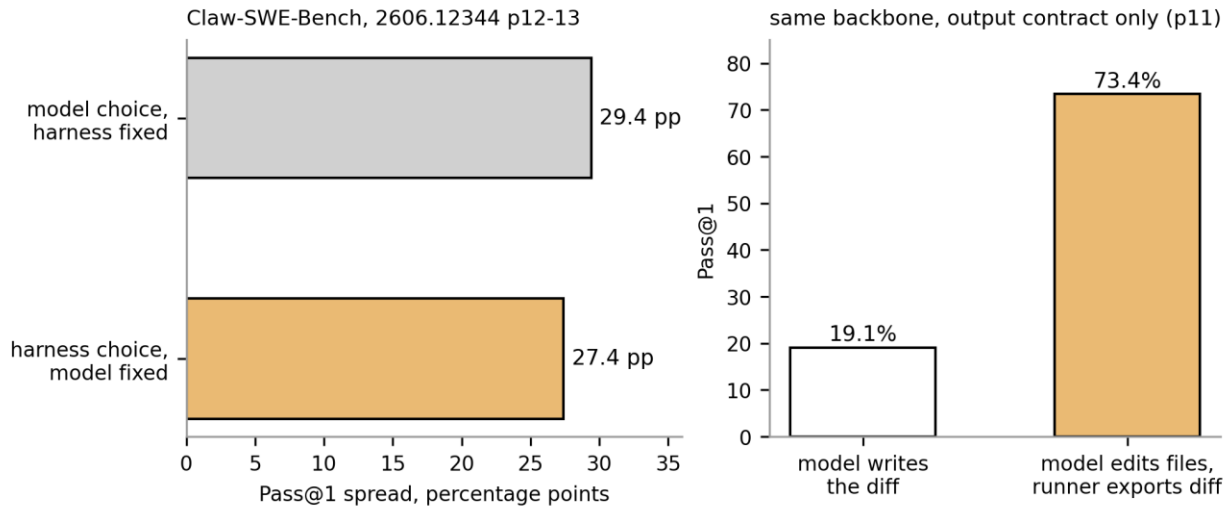


Fig. 8. At a fixed model, harness choice moves pass rates about as much as model choice (Claw-SWE-Bench, 2606.12344 p11-13).

Claw-SWE-Bench (2606.12344) measured how much the harness around a model matters on coding tasks (Fig. 8). At a fixed model, the choice of harness moves the pass rate by 27.4 points, against 29.4 for the choice of model at a fixed harness (p1), and the biggest single lever is the output contract: the same model passes 19.1% of tasks when it writes a diff and 73.4% when it edits files and the runner exports the diff (p1, p11). The effect shrinks with stronger models, from 27.4 points on Qwen 3.6-flash to 12.5 on GLM 5.1 (p3), it covers coding only, and no harness-controlled measurement exists for research work.

5.2 What dsh is: a plugin harness with no privileged core

dsh is a plugin harness. In its own words, "There is no privileged core to patch: you extend dsh by mounting a plugin beside the others, and registrations are effects that unwind when their plugin unloads." The model adapter, tool registry, session log and agent loop are all plugins on a shared context called Cordis. It is a developer preview at version 0.1.5-rc.2, and 4 of its properties matter here: substitutability, since a service boundary is made of a definition, a provider and a consumer; replayability, since anything that reaches a model request must be reconstructable from the log; immutable data, since a write publishes a version-named successor; and containment, since a sandbox service keeps policy separate from mechanism.

5.3 Stage mapping: what dsh provides at each stage, and what AI4Research adds

Table 7 puts each stage of the workflow beside the dsh mechanism that would carry it. Nothing in its third column is built.

AI4Research stage	dsh mechanism	What AI4Research would add
Compiled intent and contract	An agent pre-step; typed session events	The intermediate artifacts and their deterministic validation
Freeze	Immutable versioned successors; history derived from the log	The plan hash with the grading rules inside
Task graph	Jobs, subagents and agent teams	The frozen graph, write-scope conflicts and the projection check
Orchestration	Swappable services in place of the file inbox; a token meter	Logical and physical operator routing as service providers
Capsules	A per-tool catalog of required services and written events	Declared effects checked before admission (capsule report)
Gates	Invariants and an approval service	Frozen evaluators and the result vocabulary
Claims	The session log as an evidence store	Evidence schemas, verdict gates and the unsupported-claims list
Security	Sandbox policy and backends	Declared effects, which turn a sandbox policy from a guess into a declaration

Table 7. What dsh provides at each stage, and what AI4Research would add.

dsh settles loading, swapping, reverting, sandboxing and logging. It does not settle the contract: what a run must satisfy, what grades it, and what a capability may do.

5.4 openJiuwen: the longer-term home for the contract layer, not the runtime

The second decision of 2026-09-11 was to add the contract layer to openJiuwen later rather than port the runtime. AI4Research carries the openJiuwen name but shares none of its code, and the 2 systems differ on every axis the contract layer touches (Table 8).

	AI4Research	openJiuwen auto_research
Contract scope	1 per run, frozen	1 per round, renegotiated
Grading rules	Inside the frozen hash	Host rules in code; paper scoring by a model rubric
Adaptation	Bounded repair before the freeze	Every round
Code generation	Pre-built runners on the native path	A model coder with retries
Claims	Cue-word extraction and verbatim grounding	Host-only numeric claim status against prior papers

Table 8. AI4Research and openJiuwen auto_research compared.

6. Where this leaves AI4Research: ahead on checking, behind on generating

AI4Research appears to be ahead of the reviewed systems in 3 ways, all design properties rather than measured results: it seals the grading rules inside the run hash, it refuses to raise a claim above insufficient when its text is not found in the cited section, and it keeps an honest result vocabulary in which not-tested and environment-blocked are not failures.

It appears to be behind in 3 ways: experiment code on the trusted path covers 1 hand-built family, claim extraction is an English-only cue-word rule with unmeasured recall, and there are no external benchmark results.

The lead is a design property, not a result. Rel(AI)Build (2606.26924) already content-hashes its specification at approval and Arbor already runs a held-out gate that only its merge tool can open (Appendix D.4 and Section 4.3), so the first external result decides whether the lead holds.

6.1 Challenges: 6 open problems, each with what would settle it

The challenges are grouped by area, with what was found, what could settle it, and the strongest objection to each.

A. Measurement and judgment

#	Challenge	Evidence	What would settle it
1	Nothing is measured externally	Every strength above is a design property read in source: the journey tests and the 14-node run are internal, and there are 0 benchmark scores. At every risk tier exactly 1 node gets a model evaluator, and results do not separate the 20 rubrics from the 23 Python gates (Section 2.6). Claim recall is unmeasured (Section 2.7). In DeepFact (2603.05912), experts working unassisted judge verifiable claims correctly 60.8% of the time (p5), so human review is not a substitute	Running TruthInsightBench end to end on 1 frozen model and reading the 6 dimension scores; recording the deciding check kind on every verdict; hand-labeling the claims of 50 paper sections and reporting the cue-word rule's recall

Table 9. Challenges in measurement and judgment.

The obvious objection is that every pre-release system lacks an external score. It does, and the internal evidence still cannot stand in, since a graph gets 1 model-evaluated node and a reader cannot tell which verdicts a model decided. The work seems small: a Terminal-Bench 2.0 adapter already sits in the repository, and TruthInsightBench would need an adapter of the same shape.

B. Claims

#	Challenge	Evidence	What would settle it
2	Claims carry provenance, not truth, and only in English	Without an experiment, the strongest verdict is partially supported at 0.85, a case-insensitive substring match against the cited section. A span that fails its re-hash is dropped silently, and the reportable set is filtered on evidence ids rather than verdicts, so weak claims become conclusions and the unsupported list stays empty (Section 2.7). The English rules sit at 2 boundaries, the request and the papers, while the repository's own operating documents are in Chinese	Adding a gate that fails a claim whose span fails its re-hash, then replaying the delivery fixture with 1 span edited; filtering the reportable set on the verdict; checking whether the 3 provenance fields reach the rendered prose; a schema-bound model extractor tested per language against a hand-verified claim list

Table 10. Challenges in claim handling.

The objection is that a provenance verdict is the honest one, and the system says so in its own fields. Those fields are the reason the design is worth keeping; the gap is what a reader of the output sees. FactReview (2604.04074) shows the alternative inside the same design: a claim whose link to its evidence cannot be established is returned as inconclusive rather than carried forward (Appendix D.7).

C. Planning and the freeze

#	Challenge	Evidence	What would settle it
3	The freeze cannot handle what it did not foresee	Intent, requirement, plan graph and capsule selection get 1 repair by default, then the sprint fails, and the reviewers are advisory, so a meaning error a reviewer found can pass (Sections 2.1 and 2.2). Every node but the single model-evaluated one is frozen with no evaluator gate, and such a node records a policy pass (Section 2.3)	Freezing a graph, then delivering for a node with no evaluator gate an output that fits its schema and contradicts its acceptance criterion, and reading the verdict recorded; declaring halt, inconclusive or human in the test policy and repeating that test
4	A contract is only as good as its compilation, and a wrong plan has no recovery	Plan defects per run are unmeasured, and an open capability such as a shell or SQL routes around typed contracts, the cause of 13 of PlanCompiler's 22 failures (Section 2.3). There is no re-planning, and a node no capsule fits fails the plan at the freeze (Section 2.2)	Replaying the 24 journey requests through planning alone at 1 frozen model and counting defects per plan, against the 0 the freeze assumes; keeping the freeze and documenting its cost against Arbor and Kosmos; the generalist capsule of the capsule report

Table 11. Challenges in planning and the freeze.

The objection is that a bounded pre-freeze loop is the point of the design, which it is. What seems not deliberate is what the freeze does with an outcome nobody wrote a rule for: the node closes as a recorded pass. 13 of 22 is PlanCompiler's number, which bounds the failure mode without measuring this system, and that missing measurement is the challenge.

D. Capsule library. Trust classes, declared versus observed effects, versioning, gap records and protected paths are covered in the capsule report (Section 3 there), so this group adds no numbered challenge here.

E. Execution and autonomy

#	Challenge	Evidence	What would settle it
5	Execution is recorded, not observed	On the CLI-agent path, only the git worktree bounds the agent: its commands, files and network access are not observed, and untrusted paper text reaches it through that shell (Section 2.5). A node's network rule is spent at planning, and nothing watches the network once the node runs. Only 21 of 79 capsule manifests declare a trust class. The coordinator substrate, which the move to dsh replaces, once ran code 28 hours older than the file on disk (Section 2.4)	Running 1 experiment node on the adapter path with an outbound request placed in the runner and reading every artifact the run retains; the move to dsh, whose sandbox service and session log make that access recordable (Section 5); model-written runners under hash-and-approve

#	Challenge	Evidence	What would settle it
6	Unattended runs do not finish	3 logged runs stopped at literature discovery after a rate-limit error, each from a different cause, and the first 2 causes are fixed. The third was retryable, and the frozen policy of 1 attempt ended the sprint and cancelled 13 downstream nodes, because no hash-bound local source was declared to continue on. The run that finished without intervention did so with 6 PDFs attached (Section 2.8)	An unattended journey test from a prompt with no attachments and a fallback to hash-bound local sources declared in the frozen contract; the same prompt with the provider forced to return a rate-limit error

Table 12. Challenges in execution and autonomy.

The objections are that the coordinator incidents are dated, which they are, and that a rate-limit error is answered by a retry. The log suggests otherwise: the failure was already marked retryable, and the frozen 1-attempt policy ended the run, so loosening that policy alone would let a run continue with no evidence behind it. The fallback would have to be a hash-bound local source declared inside the contract.

6.2 Directions (proposed): the next step each challenge points to

None of these is built:

- Moving the workflow onto dsh, keeping the contract layer as the contribution (challenge 5)
- Having a model write experiment runners under hash-and-approve (challenge 5)
- Counting plan defects per run before deciding whether the freeze keeps its cost (challenge 4)
- A schema-bound model extractor at both text boundaries, with deterministic checks after it and a gate on unverified spans, tested per language (challenges 1 and 2)
- Declaring in the test policy what a node does when its frozen rules do not cover the outcome (challenge 3)
- Recording on every verdict which kind of check decided it (challenge 1)
- A hash-bound local-source fallback for retrieval nodes (challenge 6)
- Running 1 external benchmark end to end before making any comparative claim (challenge 1)

7. Open questions: 2 decisions taken, 3 decisions waiting

Every row of Table 13 names the decision it asks for.

#	Question	The decision it asks for	Owner
Q-G1	Which platform the workflow runs on	Taken: the DeepSeek Harness for now, as the more certain of the 2 candidates (Section 5)	presenter
Q-G2	Whether to port the runtime to openJiuwen or add the contract layer to it later	Taken: add the layer later, and keep the workflow on the DeepSeek Harness (Section 5.4)	presenter
Q-G3	Whether openJiuwen's auto_research is named as the closest comparison	Approve naming it in anything that leaves the team, or let Arbor stand as the closest comparison on checking (Sections 4.3 and 4.4)	supervisor
Q-G4	Which external benchmark comes before circulating	Approve TruthInsightBench and the adapter work it needs, or name a benchmark the existing Terminal-Bench 2.0 adapter already reaches, and set the date it runs (Section 3.2)	supervisor

#	Question	The decision it asks for	Owner
Q-G5	Who owns each challenge, and in what order they are taken	Assign challenges 1 and 4 for the next cycle, since both are measurements rather than builds and both gate the comparative claims (Section 6.1)	supervisor

Table 13. Open questions, the decision each asks for, and its owner.

Q-G4 unblocks the most, since every comparative claim in Sections 4 and 6 rests on a design property until an external result exists.

Appendix A. Sources

Code was read in the source of AI4Research (the intent gateway and compiler, the requirement compiler, the planner and plan compiler, the execution authority, the dispatcher and scheduler, the gate executor, the operator registry adapter, the evidence and delivery operators, the claim verdict gate, the checks registry, the operator and capsule registries, the experiment capsule, the stabilization log, the journey test report and the planner and delivery tests), of DeepSeek Harness (its architecture document and agent notes) and of openJiuwen agent-core, each at 1 pinned commit. The annotated variant of this report names every file and line. The companion report cited as the capsule report is "Capability Capsule in AI4Research", draft of 14 September 2026.

18 sources were read in full, and each is used for 1 job (Table A1).

Role	Source	Used for
Reference system	2504.08066 AI Scientist-v2	Generation-first design; 57% overlap missed by review
Reference system	2511.02824 Kosmos	Scale; 79.4% and 57.9% statement accuracy
Reference system	2606.11926 Arbor	Held-out merge gate enforced by the merge tool; 77.36 against 71.70
Reference system	openJiuwen auto_research (source code, no paper)	Per-round contract
Considered, not compared	2502.18864 Co-Scientist, 2505.13400 Robin, 2501.04227 Agent Laboratory, 2606.15497 Towards End-to-End Automation of AI Research	Why the comparator set is these 4 systems
Mechanism	2604.13092 PlanCompiler	Typed plan compilation; 44/50 against 20/50; constraint evasion
Mechanism	2503.18813 CaMeL	Control flow fixed before data; 77% against 84%
Mechanism	2606.26924 Rel(AI)Build	Specification hashed; test linkage cooperative
Mechanism	2606.19819 CREDENCE	Price of rule-based extraction, 0.811 against 0.88 to 0.91
Mechanism	2604.04074 FactReview	Closest verdict design; 79.7% recall; 17.0%
Measurement	2608.05179 verification-gap survey	Ladder rungs; pre-committed specification
Measurement	2411.14199 OpenScholar	78% to 90% fabricated citations
Measurement	2603.05912 DeepFact	27.0% unsupported; experts 60.8%; 83.4%
Measurement	2606.12344 Claw-SWE-Bench	Harness 27.4 against model 29.4 points; 19.1% against 73.4%
Benchmark	2609.05079 TruthInsightBench	40 blind tasks over 6 dimensions; 58.4 to 60.3 plateau

Table A1. The 18 sources read in full, and what each is used for.

Appendix B. Questions a reviewer will ask

Question	Answer	Section
Does it verify claims?	Without an experiment, no. It certifies that the sentence occurs in the cited section (partially supported, 0.85) and records that limitation	2.7

Question	Answer	Section
Is the span hash enforced?	Spans are re-hashed and filtered, but no gate fails a claim whose span fails. A gate on the span flag is proposed	2.7
Does a model decide anything after the freeze?	Not scheduling, routing or admission. Models execute inside capsules, and 20 of 43 registered checks are model-judged rubrics	2.4, 2.6
Can the model overrule a rejection?	No. A fidelity objection is demoted to a warning once the 1-repair budget is spent, by design, so a review model never becomes a second admission gate	2.1
What happens when no capsule fits?	The node is marked unsatisfiable, and the plan fails at the freeze. There is no generalist fallback today, and the capsule report proposes a generalist capsule	2.2
Does an LLM write experiment code?	Yes, on the adapter path through a coding agent, classed fixture-or-adapter-only. The trusted path has 1 hand-written family	2.8
Is there any external benchmark?	None. TruthInsightBench comes first, reported per dimension	3.2
Does any other system seal its grading rules inside the run hash?	None of the reviewed systems does, after a targeted search on 2026-09-13. Rel(AI)Build hashes its specification, but its test linkage is cooperative and checked after the fact	2.3
Does the hash stop tampering?	No. It detects an edit after the fact, wherever something recomputes it, and it authenticates no author	2.3
What does the freeze cost?	No re-planning when execution shows the plan was wrong, and search only inside operators. It fits execution and grading, not exploration	2.3
Is the frozen chain fully recomputed?	3 stages are recomputed, and the rest are compared by hash	2.3
What happens when a frozen rule is undefined on the outcome?	The node closes as a recorded policy pass. At every risk tier 1 node of the graph carries a model evaluator, every other node is frozen with no evaluator gate, and such a node passes on its proof obligations and artifact schema alone. A halt, inconclusive or human policy in the test policy is proposed	2.3, 6.1
Every pre-release system lacks an external score. Why does it matter here?	Because the argument is a design comparison, so until 1 external result exists no quantity can be set beside Arbor or Kosmos. The internal evidence cannot stand in: a graph gets 1 model-evaluated node, and a report reader cannot tell which verdicts a model decided	6.1, 2.6
Provenance rather than truth is deliberate. Where is the problem?	In what the output shows. A claim judged insufficient arrives as a conclusion, the unsupported-claims list is empty whatever the verdicts were, and no check requires the 3 provenance fields to reach the rendered prose. FactReview returns such a claim as inconclusive instead	2.7, 6.1, D.7
Bounded repair and advisory reviewers are by design. What is left?	The default evaluator gate. A node outside the single evaluated node is frozen with no evaluator gate and 0 repair attempts, so an outcome nobody wrote a rule for closes as a pass	2.3, 6.1
13 of 22 is PlanCompiler's number. Why does it count here?	It bounds the failure mode without measuring this system, and that missing measurement is the challenge. Plan defects per run are counted by nobody today	2.3, 6.1

Question	Answer	Section
The stale coordinator is an old incident. Is that challenge still live?	The dated incidents belong to the coordinator substrate the move to dsh replaces. The live part is the CLI-agent path used today, whose network rule is spent on filtering capsules at planning and observed nowhere at run time	2.2, 2.5, 6.1
HTTP 429 is a provider flake. Why not retry?	The provider failure was marked retryable; the frozen policy of 1 attempt ended the run and cancelled 13 downstream nodes. Loosening that policy alone would let a run continue with no evidence behind it, so the fallback has to be a hash-bound local source declared in the contract	6.1
Why move platforms?	Harness choice moves pass rates by 27.4 points at a fixed model, and the current substrate produced a coordinator running 28-hour-old code	5.1
Why not openJiuwen now?	No code is shared today, and its auto_research renegotiates its contract every round. The decision is to contribute the layer later	5.4
Is the 1-sentence design rule measured?	No. It is a translated target statement from a Chinese source	1
What does a run cost, and which models does it use?	No source records the token cost, money or wall-clock time of a sprint, and no model is named for AI4Research anywhere in this report. 102 physical operators are defined and 63 enabled	1, 3.1

Table B1. Questions a reviewer is likely to ask, with short answers.

Appendix C. Where the code evidence is

Every code claim in this report was read in source at a pinned commit of AI4Research, DeepSeek Harness or openJiuwen agent-core. The file and line behind each claim, and the excerpts that show the freeze writing its grading rules into the scheduler input, a real frozen node with its bound capsule and evaluators, and the tamper test, are in the annotated variant of this report, kept beside this one. They are omitted here so that the text reads without them.

Appendix D. Workflow details

D.1 Compiled intent

The first stage produces 4 artifacts, and each has a different author (Table D1).

Artifact	Question it answers	Author
IntentIR	What was asked?	A model
Validation	Is it structurally sound?	Deterministic code, with 7 kinds of check
Fidelity review	Does it reflect the request?	A second, independent model call
Acceptance	May the run proceed?	A deterministic gate

Table D1. The artifacts of the intent stage.

The IntentIR records 6 things:

- The goals
- The outcomes, each typed as information, an artifact or an action
- The constraints, each with a category, a statement and a machine-readable expression

- The ambiguities, each marked as blocking or not
- The conflicts, each marked for clarification or rejection
- The unknowns

Besides binding quotes and bounding repair (Section 2.1), code keeps the model in check in 2 more ways:

- Limiting the expression language to 21 operators with a fixed arity and shape. There is no subset-of operator and no universal quantifier, so a relation the language cannot express stays a literal string
- Checking a refusal twice. The intake stops the pipeline, and the handoff to the requirement stage re-verifies the decision id and its hash

The stage ends in 1 of 3 outcomes: accepted, needs clarification, where a blocking ambiguity or conflict goes back to the user, or failed.

D.2 Requirement contract

The model fills in the requirements, assumptions, discovery plan, selection authority and delivery manifest. Each requirement row carries 4 fields:

- A priority: must, should or may
- An acceptance rule, with its kind and required values
- A check, named by its id in the checks registry
- A semantic role: research scope, process, delivery, outcome, resolution or constraint

Deterministic code then looks for defects, including 3 kinds:

- A changed constraint. Intent constraints must be copied byte for byte, so a model that flips "must not" to "must" is caught
- Missing coverage. Every Intent id must be referenced by some requirement, or the contract fails
- An unknown check, together with rules on named-file checks, roles and coverage

The stage makes at most 3 model calls and 1 structural repair. A requirement reviewer comments on the contract, but its comments are advisory, and the admission authority is recorded as deterministic contract validation. The checks registry holds 2 kinds of check (Table D2).

Kind	Count	Applies to	Decided by
Deterministic	23	An artifact type, applied automatically	A Python gate
Semantic	20	An output that a requirement names	A rubric, judged by a model

Table D2. The 2 kinds of registered check.

D.3 Plan and capsule binding

The plan is built in 8 steps:

1. Catalog snapshot. Code takes a snapshot of the workflows, logical operators, capsule summaries and enabled physical operators and hashes it, so the plan records exactly what it was allowed to choose from
2. Planning decision. The planner chooses between a direct response, exact reuse of a known workflow, or a new plan
3. Plan graph. The model lists the nodes and the dependencies of each, and the edges of the graph are exactly that list. Code rejects unknown dependencies, self-dependencies, cycles, a node that reads an artifact whose producer is not an ancestor, requirements no node covers, and more than 1 model-evaluator node. 1 repair is allowed, and a second failure fails the sprint

4. Hard capsule filter. Before any model sees the candidates, code excludes capsules for 13 reasons
5. Selection. The model picks only among the capsules that survived the filter, and code derives the dispatch type and validates the choice 8 ways. A second model reviews the fit, but a failed review only produces a warning. 1 repair is allowed
6. Composition. When no single capsule fits, capsules may be chained, but only through registered adapters
7. Node binding. The capsule id, proof obligations, artifact types, effect union and approved physical operators are copied from the manifest, never from the model
8. Physical plan. Each node receives frozen execution candidates, excluding operators with the wrong role or task class, operators the capsule forbids, and providers that are unauthenticated or not idle

D.4 The freeze

The freeze runs in 8 steps of deterministic code, and any step can fail the sprint:

1. The evaluation budget becomes a test policy, which sets the risk tier and the number of model-evaluator nodes, 1 or, for a rapid smoke run, 0
2. The capsule plan and the physical plan are validated with their adapters and merged into 1 graph
3. The plan is validated and stamped with a certificate
4. The evaluation plan is compiled and validated
5. The scheduler input, meaning the exact graph the scheduler will run, is compiled
6. The live capsule and operator definitions are checked, and if any definition changed during planning, the freeze aborts
7. The frozen run contract is written with a reference to every upstream artifact and sealed with its own hash
8. The plan is accepted and handed to the runtime

An existing test tampers with a frozen bundle in 3 ways, and verification catches each one (Table D3).

Tamper	Caught by
Set a node's priority to 999 in the scheduler input	Recompute: the recomputed scheduler input no longer matches
Edit the acceptance criteria in the evaluation plan	Reference hash: the evaluation plan no longer matches its hash
Edit the rationale in the composition selection	Reference hash: the composition selection no longer matches its hash

Table D3. The 3 tampers the existing test applies, and how verification catches each.

Plan nodes also have no half-bound state: a selected node must carry its capsule, dispatch type, role, artifact types, effect union and proof obligations, and an unselected node must be marked unsatisfiable with no stages.

Other work fixes parts of a run before execution, but none fixes the grading rules with the plan (Table D4).

Family	Example	Fixed before execution	Not fixed
Plan-then-execute security	CaMeL (2503.18813)	Control flow, before untrusted data is read; 77% of AgentDojo tasks solved with provable security against 84% undefended (p1)	Success criteria

Family	Example	Fixed before execution	Not fixed
Compiled pipelines	PlanCompiler (2604.13092)	A statically validated typed graph	Evaluation policy
Deterministic control planes	Rel(AI)Build (2606.26924)	The specification, content-hashed at approval, plus install gates, blocklists and phase transitions	Which files and tests satisfy each criterion, which the agent declares during implementation, "cooperative and auditable post-hoc, not pre-execution enforced" (p1, p20)
Research agents	AI Scientist-v2, Kosmos, Arbor, openJiuwen auto_research	Nothing global; plans evolve every cycle or round	The plan and the rules that grade it

Table D4. What other work fixes before execution.

D.5 Dispatch

1 tick of the dispatcher takes 6 steps (Fig. D1):

1. Taking the graph lock, and stopping if the runtime handoff is forbidden or a guard fails
2. Reconciling the dispatches of earlier ticks
3. Applying the approval gate and the terminal barrier, which closes a finished sprint
4. Verifying the runtime projection, meaning that every frozen field of the runtime graph still matches the contract, and computing the set of ready nodes
5. Admitting and leasing the selected nodes, and writing a work envelope into each operator's inbox
6. Saving the ledger with its revision increased by 1

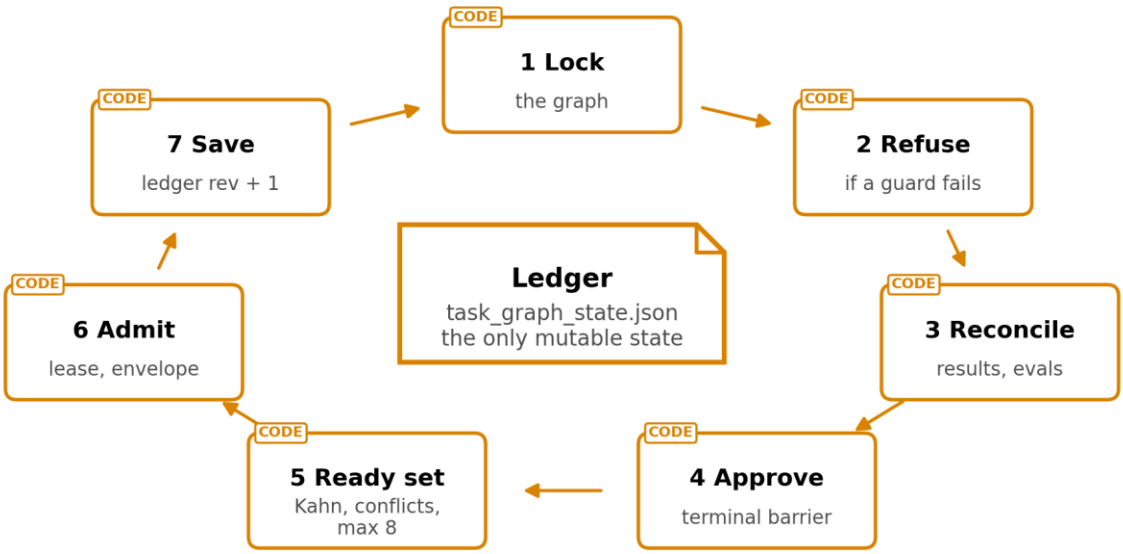


Fig. D1. 1 dispatch tick around the ledger, the only mutable state.

The rules a tick enforces are fixed for the whole run (Table D5).

Mechanism	Rule
Scheduling	Kahn topological order; a cycle is an error

Mechanism	Rule
Write safety	Nodes with overlapping declared write scopes never run together, and a node that declares no write scope is treated as an exclusive writer
Projection check	Every frozen field is compared with the runtime node, and only keys on a 54-name allowlist may change
Failure	A failed evaluation leads to repair with the findings, up to the repair budget, and then to stopping the node or failing the run; dependents are skipped
Monotonic progress	A passed node never regresses
Paid work	Work envelopes are fingerprinted, so a retry never repeats a paid call
Evidence	A receipt is not evidence; a node closes only on the chain of result, callback and an evaluator PASS

Table D5. The dispatch mechanisms and their rules.

Node statuses fall into 3 groups, plus a separate status for nodes that need human review. Ready statuses are pending, queued, blocked, worker-blocked and failed review. Active statuses are assigned, dispatched, in progress, running and reviewing. Terminal statuses are passed, failed, skipped, cancelled, and skipped because a parent passed.

D.6 Operator execution paths

The 2 paths carry the same node contract and differ most in what is observed (Table D6).

	Registry path	CLI-agent path
Before running	Code re-verifies the graph projection, that the operator is among the frozen candidates, the exact artifact contract, the routes, the capsule binding and the write scope, and checks the registry, node kind and implementation against an allowlist	The agent receives a terminal pane in a git worktree, the dispatch text and injected research rules
Model calls	The request, schema, response, events and hashes of each call are kept	Whatever the CLI agent does
Output	A typed result file, always	The exit code, converted into a result file
Observed	Inputs, outputs and model calls	Nothing: commands, files and network access are not observed
Trust class	Set per capsule	Fixture-or-adapter-only

Table D6. The 2 operator paths compared.

D.7 Claim handling compared with published work

2 published systems show the price of this design and the nearest alternative:

- CREDENCE (2606.19819) prices rule-based extraction. Its rule-only baseline, spaCy sentence segmentation, reaches 0.811 Semantic-F1 against 0.88 to 0.91 for model decomposition, loses no entities, and leaves atomicity violations near their maximum at 0.52 to 0.56, because "rule-based splitting cannot resolve complex multi-clause structures" (p22-23). AI4Research's cue-word rule is cruder than that baseline
- FactReview (2604.04074) is the closest verdict design, with a closed set of 4 outcomes, supported, partially supported, in conflict and inconclusive, and an inconclusive verdict whenever the link between claim and evidence cannot be established (p2). It extracts claims with a model, at 79.7% recall and 84.3% F1 (p5), and removing execution evidence changes 17.0% of claim statuses, more than removing any other source (p1, p6)

Grounding matters because ungrounded claims are common, and human checking is a weak fallback (Fig. D2):

- OpenScholar (2411.14199) reports that GPT-4o fabricates citations 78% to 90% of the time without retrieval (p1)
- DeepFact (2603.05912) finds that 27.0% of claims in deep-research reports are unsupported (p8)
- In DeepFact, PhD-level experts working unassisted judge verifiable claims correctly 60.8% of the time (p5), while the best automated checker reaches 83.4% and earlier fact-checking pipelines at most 58.5% (p9)

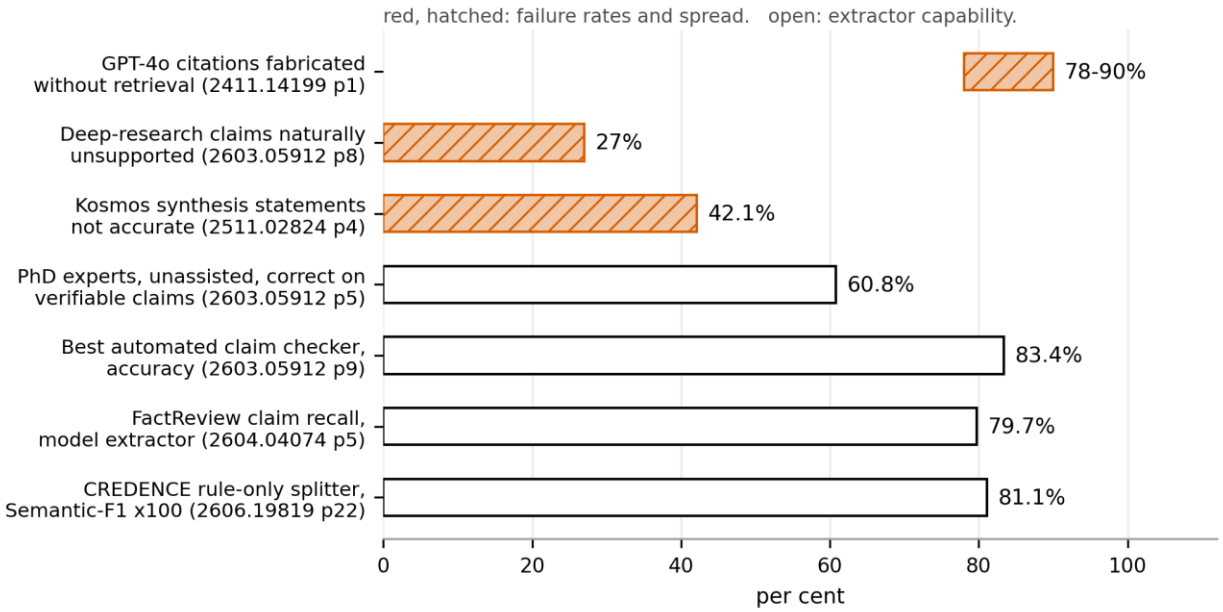


Fig. D2. Red, hatched: measured failure rates. Open: measured checker and extractor capability.

D.8 Testing layers

AI4Research separates 3 layers of testing, and only 1 decides acceptance (Table D7).

Layer	Content	Authority
Journey tests	24 real tasks run through the production entry points	The only acceptance evidence
Regression suites	The existing tests	Locate faults, never decide the verdict
Atomic matrix	1,502 reviewed rows over 142 Level-2 features	Diagnostic

Table D7. The 3 layers of testing.